

DEVELOPER SETUP GUIDE

everything-claude-code + BMAD-METHOD

A beginner-friendly, step-by-step guide to setting up both frameworks for AI-assisted development.

March 2026 | Trivium eSolutions

Companion to: ECC vs BMAD Comparative Analysis Report

Prerequisites at a Glance

Node.js >= 20 | Claude Code >= v2.1 | Git
Anthropic account (Pro, Team, Enterprise, or API)
Terminal: macOS / Linux / Windows (WSL recommended)

0. What This Guide Covers

This guide walks you through setting up **both** everything-claude-code (ECC) and BMAD-METHOD on your machine, configuring them to work together, and running your first AI-assisted development session. No prior experience with either tool is assumed.

Section	Topic	Description
1	Prerequisites	What you need before starting
2	Install Claude Code	Getting Claude Code running on your machine
3	Install everything-claude-code	Adding ECC's agents, hooks, rules, and skills
4	Configure ECC	Hook profiles, model routing, and environment setup
5	Install BMAD-METHOD	Adding BMAD's structured workflow system
6	Configure BMAD	Agent customization and track selection
7	Integrate Both Together	Making ECC + BMAD work as a unified system
8	Autopilot Mode	Hands-free orchestration — no slash commands needed
9	Your First Session	A walkthrough of a complete development session
10	Quick Reference	Cheat sheet of essential commands
11	Troubleshooting	Common issues and fixes

1. Prerequisites

Before installing either tool, make sure you have the following:

Requirement	Minimum Version	How to Check
Node.js	>= 20.0.0	node --version
npm	Included with Node.js 20+	npm --version
Git	Any recent version	git --version
Claude Code	>= v2.1.0	claude --version
Anthropic Account	Pro, Team, Enterprise, or API access	claude (on first launch)
Operating System	macOS, Linux, or Windows (WSL)	uname -a

Installing Node.js (if not already installed):

```
# macOS (using Homebrew)
brew install node

# Ubuntu / Debian
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs

# Or use nvm (Node Version Manager) - recommended
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.0/install.sh | bash
nvm install 20
nvm use 20
```

Authentication — Choose Your Path:

Claude Code supports multiple authentication methods. Pick the one that matches your Anthropic subscription:

Plan	How to Authenticate	Notes
Pro (Individual)	Browser-based login on first launch of claude	No API key needed. Usage included in subscription.
Team	Browser-based login on first launch of claude	No API key needed. Admin manages seats via console.
Enterprise	SSO via your org's identity provider	No API key needed. Configured by your IT admin.
API / Direct Access	Set ANTHROPIC_API_KEY environment variable	Pay-as-you-go billing. Key from console.anthropic.com

For API users only — setting your key:

```
# Only needed if you're using API/direct access (not Pro, Team, or Enterprise)

# Add to your shell profile (~/.bashrc, ~/.zshrc, etc.)
export ANTHROPIC_API_KEY="sk-ant-your-key-here"

# Then reload your shell
source ~/.zshrc # or ~/.bashrc
```

Tip: Most users on Pro or Team plans can skip the API key step entirely. Just run "claude" and authenticate through your browser when prompted.

2. Install Claude Code

Claude Code is the command-line AI coding agent from Anthropic. Both ECC and BMAD-METHOD are built to work with it.

Step 1: Install Claude Code globally

```
npm install -g @anthropic-ai/claude-code
```

Step 2: Verify the installation

```
claude --version  
# Should show v2.1.0 or higher
```

Step 3: Run Claude Code for the first time

```
# Navigate to your project directory  
cd ~/my-project  
  
# Launch Claude Code  
claude
```

On first launch, Claude Code will ask you to authenticate. Follow the prompts to connect your Anthropic account.

Step 4: Understand the key directories

Directory	Purpose
~/claude/	Global config (settings, rules, skills that apply to ALL projects)
.claude/ (in project root)	Project-level config (rules, skills specific to THIS project)
~/claude/settings.json	Global settings including environment variables
.claude/settings.json	Project-level settings

Tip: ECC recommends installing at the global level (~/claude/) so its optimizations apply to all your projects.

3. Install everything-claude-code

ECC adds 28 agents, ~116 skills, ~59 commands, lifecycle hooks, and a self-learning system to your Claude Code setup. There are multiple installation methods — pick the one that suits you best.

Method A: Claude Code Plugin (Recommended)

The simplest approach. Run these commands inside a Claude Code session:

```
# Inside a Claude Code session, run:
/plugin marketplace add affaan-m/everything-claude-code
/plugin install everything-claude-code@everything-claude-code
```

Method B: Git Clone + Install Script

For more control over what gets installed:

```
# Clone the repository
git clone https://github.com/affaan-m/everything-claude-code.git
cd everything-claude-code

# Install dependencies
npm install

# Run the install script with your primary language(s)
./install.sh typescript          # Single language
./install.sh typescript python go # Multiple languages
```

Method C: NPM Package + Selective Install

```
npm install @affaan-m/everything-claude-code
npx ecc-install

# Or with specific profiles:
ecc install --profile developer \
  --with lang:typescript \
  --with agent:security-reviewer \
  --without skill:continuous-learning
```

Tip: The interactive installer (ecc-install) will ask whether to install at user-level (~/.claude/), project-level (.claude/), or both. For beginners, user-level is recommended.

Step : Verify ECC installation

```
# Inside Claude Code, run the doctor command:
/doctor

# You should see agents, skills, and hooks loaded
```

4. Configure everything-claude-code

4.1 Choose Your Hook Profile

ECC hooks intercept agent behavior in real time. Choose a profile based on your comfort level:

Profile	What It Does	Best For
minimal	Essential safety hooks only (block --no-verify, config protection)	Beginners, exploration
standard	Balanced quality + safety (auto-format, type-check, quality gates)	Day-to-day development (DEFAULT)
strict	All hooks + extra reminders (stricter guardrails, more checks)	Production/security-critical work

```
# Set your profile (add to ~/.bashrc or ~/.zshrc)
export ECC_HOOK_PROFILE=standard

# Optionally disable specific hooks you don't need:
export ECC_DISABLED_HOOKS="pre:bash:tmux-reminder,post:edit:typecheck"
```

4.2 Set Up Model Routing (Cost Optimization)

This is one of ECC's most powerful features. Route different types of work to different Claude models to save cost without sacrificing quality.

Model	Cost	Use For
Haiku	~80% cheaper than Opus	File reading, exploration, test running (via subagents)
Sonnet	~60% cheaper than Opus	Standard coding, ~80% of tasks (DEFAULT)
Opus	Full price	Complex architecture, security-critical code, when first attempt failed

Add this to your `~/.claude/settings.json`:

```
{
  "env": {
    "CLAUDE_CODE_SUBAGENT_MODEL": "anthropic/claude-haiku-4-5",
    "MAX_THINKING_TOKENS": "10000",
    "CLAUDE_AUTOCOMPACT_PCT_OVERRIDE": "50"
  }
}
```

Tip: This routes subagent work (file reads, tests) to Haiku while your main session stays on Sonnet. Opus is manually selected for complex tasks.

4.3 Enable Continuous Learning (Instincts)

ECC's Instincts system automatically learns from your coding sessions. It captures error resolutions, user corrections, workarounds, and project-specific patterns.

How it works:

- Hooks observe your sessions automatically (via PreCompact and Stop hooks)
- Patterns are extracted into "instincts" with confidence scores (0.3–0.9)
- Instincts are stored in `~/.claude/skills/learned/`
- Over time, instincts can evolve into skills via the `/evolve` command

Tip: This is enabled by default with the standard hook profile. For the best results, maintain session quality from the start — instincts learn from what you do.

5. Install BMAD-METHOD

BMAD-METHOD adds a complete agile workflow with AI agent personas. It guides you from brainstorming through architecture to sprint-based implementation.

Step 1: Run the installer

```
# Navigate to your project root
cd ~/my-project

# Run the BMAD installer
npx bmad-method install
```

Step 2: Follow the interactive setup

The installer presents an interactive terminal UI:

- **Location Selection** — Choose where to install (usually the current directory)
- **AI Tool Selection** — Select "Claude Code" (use arrow keys + SPACE to toggle, ENTER to confirm)
- **Module Selection** — Choose "BMM" (the default/standard module) for most projects
- **Configuration** — Answer any follow-up prompts based on your selected module

Step 3: Verify the installation

```
# Check that the BMAD directories were created
ls _bmad/
# You should see: agents/ workflows/ tasks/ _config/ _memory/ core/

# Check Claude Code integration files
ls .claude/commands/bmad/
ls .claude/skills/bmad/
```

Step 4: Test with bmad-help

```
# Inside Claude Code, run:
/bmad-help what should I do first?
```

Tip: bmad-help inspects your project and recommends the next workflow. It's your always-available guide.

What the installer creates:

Directory/File	Purpose
_bmad/agents/	Compiled agent persona files (Mary, Winston, etc.)
_bmad/workflows/	Workflow step files for each phase
_bmad/tasks/	Standalone task definitions
_bmad/_config/config.yaml	Your BMAD configuration

Directory/File	Purpose
<code>_bmad/_config/customizations/</code>	Agent customization overrides
<code>_bmad/_memory/</code>	Agent sidecar folders for persistence between sessions
<code>_bmad-output/</code>	Generated artifacts (PRDs, architecture docs, stories)
<code>.claude/commands/bmad/</code>	Slash commands for Claude Code
<code>.claude/skills/bmad/</code>	BMAD skills for Claude Code

6. Configure BMAD-METHOD

6.1 Choose Your Planning Track

You don't set this during installation — you select it when you start your first workflow. Here's how to choose:

Track	Team Size	Timeline	When to Use
Quick Flow	1–2 devs	Hours to days	Small features, experienced teams, rapid prototyping
BMad Method	3–7 people	Weeks to months	New products, full lifecycle planning needed
Enterprise	8+ teams	Months+	Compliance requirements, multi-tenant, governance

6.2 Customize Agent Personas (Optional)

Each BMAD agent has a persona you can customize. This is optional but useful if you want agents to match your team's style.

```
# Create a customization file for the PM agent
# Location: _bmad/_config/customizations/bmm/agents/pm.customize.yaml

persona:
  role: "Senior Product Manager"
  identity: "Expert in SaaS and B2B products"
  communication_style: "Direct, strategic, data-driven"
  principles:
    - "Always validate with user research"
    - "Balance features with time to market"
```

Tip: Customization files survive BMAD updates — the core agent files get overwritten but your .customize.yaml files are preserved.

6.3 Meet the Key Agents

Agent	Name	Role
Analyst	Mary	Market research, domain analysis, product briefs
PM	John	PRD creation, requirements elicitation, prioritization
Architect	Winston	System design, technology decisions, architecture docs
Scrum Master	Bob	Sprint planning, story creation, retrospectives

Agent	Name	Role
Developer	Amelia	Implementation, TDD, code writing
QA	Quinn	Testing strategy, quality assurance, validation
UX Designer	Sally	User experience design, wireframes, user flows
Tech Writer	Paige	Documentation, runbooks, onboarding guides
Quick Flow	Barry	Simplified end-to-end for small projects

7. Integrate Both Together

This is where the magic happens. ECC and BMAD operate at different layers and complement each other. Here's how to make them work as a unified system.

7.1 Understand the Layer Model

Layer	Handled By	What It Does
Methodology (what to build, when)	BMAD-METHOD	Brainstorming, PRDs, architecture, story planning, code review
Agent Behavior (how the AI executes)	ECC	Hooks, rules, security scanning, self-learning, model routing

7.2 Resolve Potential Conflicts

Both tools create files in the `.claude/` directory. Here's how to avoid collisions:

Potential Conflict	Solution
Skills directory collision (both use <code>.claude/skills/</code>)	BMAD installs to <code>.claude/skills/bmad/</code> ECC installs to <code>.claude/skills/</code> No collision by default
Both have TDD workflows	Pick one and stick with it. Recommend: ECC for test execution, BMAD for test planning
Fresh chat (BMAD) vs. long sessions (ECC learning)	Use fresh chats for BMAD planning phases. Use longer sessions during ECC-guided implementation for instinct capture.
Rules overlap	ECC's rules are language-specific. BMAD's workflows are process-specific. They generally don't conflict.

7.3 Recommended Combined Workflow

Phase	Tool	What You Do
1. Brainstorm & Analysis	BMAD	Use Mary (Analyst) for research. New chat for each workflow.
2. Planning & Requirements	BMAD	Use John (PM) for PRD creation. Socratic questioning refines ideas.
3. Architecture	BMAD + ECC rules	Use Winston (Architect) for design. ECC's architecture rules guide quality.

Phase	Tool	What You Do
4. Story Creation & Sprint Planning	BMAD	Use Bob (Scrum Master). Stories flow into _bmad-output/
5. Implementation	ECC (primary)	ECC hooks auto-format, type-check, security-scan. Instincts learn patterns.
6. Code Review	Both	BMAD's Quinn for QA workflow. ECC's code-reviewer agent for specifics.
7. Retrospective	BMAD	Sprint retrospective workflow. Capture learnings for next sprint.

8. Autopilot Mode — Hands-Free Orchestration

The integration above requires you to know which tool to invoke at each phase. **Autopilot mode** eliminates that — Claude automatically detects your project phase and routes to BMAD or ECC mode without you typing any slash commands.

8.1 How Autopilot Works

Three mechanisms work together to create automatic routing:

Mechanism	What It Does	When It Runs
SessionStart Hook	Runs a shell script that scans your project for BMAD artifacts (PRDs, stories, etc.) and source code	Every time you launch claude or resume a session
CLAUDE.md	Contains conditional routing instructions that Claude follows based on the detected phase	Loaded at session start; always in Claude's context
Orchestration Rules	Defines artifact naming conventions and conflict resolution so the detection stays accurate	Always active; path-based activation

8.2 The Phase Detection Flow

The SessionStart hook inspects your project and determines which lifecycle phase you're in:

Detected Phase	What Triggers It	Claude's Auto-Behavior
IDEATION	No BMAD artifacts, no source code	Enters BMAD mode as Mary (Analyst). Starts brainstorming and research.
ANALYSIS	BMAD dir exists but no product brief or PRD	Continues BMAD analysis. Guides toward product brief.
PLANNING	Product brief exists, no PRD yet	BMAD mode as John (PM). Guides PRD creation via elicitation.
SOLUTIONING	PRD exists, no architecture doc	BMAD mode as Winston (Architect). Guides system design.
STORY_CREATION	PRD + architecture exist, no stories	BMAD mode as Bob (Scrum Master). Breaks down into epics and stories.
READY_TO_BUILD	Stories exist, no source code yet	Switches to ECC mode. ECC hooks, agents, and rules active.
BUILDING	Stories + source code both exist	ECC mode. References stories for acceptance criteria.

Detected Phase	What Triggers It	Claude's Auto-Behavior
BROWNFIELD	Source code exists but no BMAD artifacts	ECC mode by default. Offers BMAD if user wants structure.

8.3 Install Autopilot

The autopilot consists of 4 files. You can install them with one command or copy them manually.

One-command install:

```
# Download and run the installer
bash install-unified.sh
```

Or install manually (4 files):

File	Location	Purpose
CLAUDE.md	Project root	Conditional routing instructions that Claude follows automatically
hooks.json	.claude/hooks.json	Registers the SessionStart hook that runs detection on every launch
detect-project-state.sh	.claude/hooks/	Shell script that scans for artifacts and outputs the lifecycle phase
unified-orchestration.md	.claude/rules/	Artifact naming conventions and conflict resolution rules

8.4 What the User Experience Looks Like

After installation, here's what a typical session looks like — no slash commands needed:

Scenario: Starting a new project

```
$ cd ~/new-project
$ claude

# Hook runs automatically, detects: PHASE=IDEATION
# Claude reads CLAUDE.md, enters BMAD mode

You: "I want to build a SaaS dashboard for tracking team metrics"

# Claude responds as BMAD's Mary (Analyst):
# "Great idea! Let me help you think through this. What kind of
# team metrics are most important to you? Let's start with..."

# After brainstorming, Claude saves: _bmad-output/product-brief.md
# Next session: hook detects PHASE=PLANNING, Claude becomes John (PM)
```

Scenario: Implementation after planning is done


```
$ claude

# Hook detects: PRD + architecture + stories all exist
# PHASE=BUILDING
# Claude automatically enters ECC mode

You: "Let's implement the user auth story"

# Claude references _bmad-output/stories/auth-story.md for criteria
# ECC hooks auto-format, type-check, security-scan
# ECC agents (code-reviewer, tdd-guide) activate on code changes
# Instincts capture patterns for future sessions
```

Tip: Autopilot is ~90% hands-free. The remaining 10% is when Claude asks you to confirm a phase transition or clarify ambiguous intent — which is a feature, not a bug.

8.5 Important: Save Artifacts Correctly

Autopilot works because the detection script checks for specific files in specific locations. The CLAUDE.md instructs Claude to save artifacts to the right places, but if you manually move files, the detection may get confused. Stick to these conventions:

```
_bmad-output/
  product-brief.md      # Phase: ANALYSIS -> PLANNING
  prd.md                # Phase: PLANNING -> SOLUTIONING
  architecture.md       # Phase: SOLUTIONING -> STORY_CREATION
  ux-design.md          # Optional
  stories/              # Phase: STORY_CREATION -> READY_TO_BUILD
    epic-name/
      story-name.md
  sprints/              # Sprint tracking
  retros/               # Retrospectives
```

9. Your First Session — A Complete Walkthrough

Let's walk through a real development session using both tools together. We'll build a small feature from idea to code.

Phase A: Plan with BMAD (New Chat)

```
# Start Claude Code in your project
cd ~/my-project
claude

# Ask BMAD for guidance
/bmad-help I want to add user authentication to my app

# BMAD will recommend a workflow based on your project state.
# For a new feature, it might suggest Quick Flow (Barry).

# Follow the guided workflow:
# Barry will walk you through spec > architecture > implementation
```

Phase B: Implement with ECC (Same or New Chat)

```
# ECC hooks activate automatically during implementation.
# You'll see them working:
#
# [PreToolUse] Checking for --no-verify flags...
# [PostToolUse] Running prettier on modified files...
# [PostToolUse] Type-checking with tsc...

# Use ECC slash commands:
/plan    # Get an implementation strategy
/tdd     # Switch to test-driven development mode
/cost    # Check your token spend
```

Phase C: Review (New Chat for BMAD)

```
# Start a fresh chat for code review
claude

# Use BMAD's QA workflow
# Quinn will review against acceptance criteria from your stories

# ECC's code-reviewer agent also runs automatically
# when code is modified, providing language-specific feedback
```

Tip: For your very first session, start simple. Use `/bmad-help` to orient yourself, and let ECC's hooks do their thing in the background. You don't need to master everything on day one.

10. Quick Reference — Cheat Sheet

ECC Essential Commands

Command	What It Does
<code>/plan</code>	Get an implementation strategy for your current task
<code>/tdd</code>	Switch to test-driven development mode
<code>/cost</code>	Check your current token usage and estimated cost
<code>/compact</code>	Manually trigger context compaction at a logical boundary
<code>/doctor</code>	Check that all ECC components are properly installed
<code>/evolve</code>	Promote learned instincts into permanent skills
<code>/clear</code>	Clear context for unrelated tasks

BMAD Essential Commands

Command	What It Does
<code>/bmad-help</code>	Context-aware guide — recommends your next workflow
<code>/bmad-party-mode</code>	Multi-agent debate session for complex problems

ECC Environment Variables

Variable	Values	Default
<code>ECC_HOOK_PROFILE</code>	minimal standard strict	standard
<code>ECC_DISABLED_HOOKS</code>	Comma-separated hook IDs	(none)
<code>CLAUDE_CODE_SUBAGENT_MODEL</code>	anthropic/claude-haiku-4-5	(main model)
<code>MAX_THINKING_TOKENS</code>	Number (e.g. 10000)	(default)
<code>CLAUDE_AUTOCOMPACT_PCT_OVERRIDE</code>	0–100	(default)

Key File Locations

Path	What It Contains
<code>~/.claude/settings.json</code>	Global settings, env vars, model routing config
<code>~/.claude/rules/</code>	ECC rules (common + language-specific)

Path	What It Contains
~/claude/skills/learned/	ECC auto-captured instincts
.claude/commands/bmad/	BMAD slash commands (project-level)
.claude/skills/bmad/	BMAD skills (project-level)
_bmad/_config/config.yaml	BMAD project configuration
_bmad-output/	BMAD-generated artifacts (PRDs, stories, etc.)

11. Troubleshooting

ECC Issues

Problem	Cause	Fix
Hooks not loading	Claude Code < v2.1, or manual install issue	Update Claude Code: <code>npm update -g @anthropic-ai/claude-code</code> Re-run ECC install
"Duplicate hooks file detected"	hooks.json referenced in plugin.json manifest	Remove hooks field from .claude-plugin/plugin.json
Plugin manifest validation errors	Agents field uses directories, not file paths	Use explicit paths: "agents": [".agents/planner.md"]
Rules not applying	Rules not copied to ~/.claude/rules/	Run <code>./install.sh <language></code> or manually copy rule directories
Instincts learning bad patterns	Poor quality initial sessions	Delete <code>~/.claude/skills/learned/</code> and start fresh with good sessions

BMAD Issues

Problem	Cause	Fix
/bmad-help not found	Skills not installed for Claude Code	Re-run: <code>npx bmad-method install</code> Select Claude Code in IDE picker
Installer hangs on Windows	TUI keyboard input issue in v6 alpha	Use non-interactive: <code>npx bmad-method install --yes --ide claude-code</code>
Missing _bmad/ directory	Install ran in wrong directory	<code>cd</code> to project root first, then re-run installer
ESM/CommonJS error during install	Node.js compatibility issue	Update Node.js to 20+: <code>nvm install 20 && nvm use 20</code>
Agent commands not appearing	IDE needs restart	Close and reopen Claude Code, or reload terminal session

General Tips

- Always check versions first: **node --version**, **claude --version**, **npm --version**
- When in doubt, re-run the installer — both tools support safe re-installation
- Keep ECC global (`~/.claude/`) and BMAD project-level (`.claude/` and `_bmad/`) to avoid conflicts

- Start with ECC's **minimal** hook profile and upgrade to **standard** once comfortable
- Use **/bmad-help** liberally — it's designed to guide you to the right next step
- Monitor costs early with **/cost** — model routing can save significant money

Guide Version: March 2026 | Based on ECC v1.8.0 and BMAD-METHOD v6.2.0 | Trivium eSolutions
Commands and file paths verified against official documentation. For the latest updates, check the respective GitHub repositories.